Coil: Compiling Homomorphic Circuits with Control Flow

Raghav Malik
School of Electrical and Computer Engineering
Purdue University
West Lafayette, IN, USA
malik22@purdue.edu

Benjamin Delaware
School of Computer Science
Purdue University
West Lafayette, IN, USA
bendy@purdue.edu

Milind Kulkarni
School of Electrical and Computer Engineering
Purdue University
West Lafayette, IN, USA
milind@purdue.edu

Abstract

Fully Homomorphic Encryption (FHE) enables evaluating computations over ciphertexts without revealing the encrypted data. A key challenge when compiling for FHE is dealing with control flow: the usual semantics require the party evaluating the program to learn something about each branch, which violates the basic premise of secure computations. Standard "multiplexer"-based compilers solve this problem by generating programs that require the evaluator to execute *every possible path*, obviating the need to know which path is correct. Thus, they provide oblivious semantics at the cost of producing unwieldy circuits that are difficult to effectively optimize.

We present Coil, an FHE compiler that replaces the use of multiplexers with *path forests*, enabling a series of optimizations that result in more efficient realizations of complex branching kernels. We demonstrate on a variety of benchmarks that Coil outperforms traditional approaches to FHE control flow, often by more than an order of magnitude.

1 Introduction

Fully Homomorphic Encryption (FHE) is an execution model that allows programs to be evaluated over both *public* (plaintext) and *private* (ciphertext) data, enabling execution on an untrusted platform (an "evaluator") while guaranteeing that private values are not leaked. While FHE represents a promising solution to problems like secure machine learning and computation offloading, most FHE schemes provide a very limited (but nevertheless complete) set of operations, requiring applications to be encoded as *arithmetic circuits* [14, 25]. Manually expressing application logic in terms of low-level arithmetic gates is tedious and nonintuitive. Much of the work in this space, therefore, focuses on developing compilers to alleviate some of this burden [1, 13, 14, 16, 18, 25, 32].

1.1 Oblivious Control Flow

One of the key challenges that FHE compilers have to deal with is the problem of *oblivious control flow*, in which control flow depends on a private value. Because FHE semantics prohibit the evaluator from examining private values, execution cannot simply follow the appropriate control flow path. Thus, FHE compilers equip programmers with a secure conditional operator that *executes every branch*, and then uses the encrypted conditions and a network of secure multiplexers ("muxes") to securely select the correct result [2, 18, 19]. While naively embedding ciphertext computation into a multiplexing network faithfully captures the private computation, doing so naively wastes work in two ways: (1) it can compute values that a plaintext version of the program would never use, and (2) it hides values that the evaluator could learn from publicly available information.

To illustrate, consider the following function[1]:

```
λ(x: int, y: int).
    let z = if x - 1 < y then 1 else 2 in
    let w = if x < y + 1 then z + 3 else z - 3 in
    [z, w]
```

If instead `x` and `y` were ciphertexts, secure evaluation would require replacing the conditionals with secure counterparts[2]:

```
λ(x: int, y: int).
    let z = mux(x - 1 < y, 1, 2) in
    let w = mux(x < y + 1, z + 3, z - 3) in
    [z, w]
```

Evaluating this program *always* computes four paths: two for `z` and two for `w`; the computation of `w` must treat `z` as an encrypted value, since it is the result of a secure `mux`. Note, however, that within each branch of `w`'s `mux`, only one value of `z` is actually possible (`1 + 3` or `2 - 3`). A smarter approach could compute the branch just once, and avoid the unnecessary arithmetic. This paper presents a compiler, Coil, that transforms the first function into a secure version that avoids both forms of wasted work.

We observe that FHE is not the only circumstance in which an evaluator attempts to consider every possible control flow path. A symbolic execution engine tries to statically explores all paths through a program to build path conditions, find bugs, or perform some other analyses using *symbolic values* which can only be determined at run-time [10]. Our key insight is that the way FHE compilers treat ciphertexts—as variables whose concrete values won't be known until

[1] This snippet is written in our surface language, shown in Figure 3.
[2] `mux(b, x, y)` is an operation that obliviously evaluates to either `x` or `y` based on the value of the encrypted condition `b`.

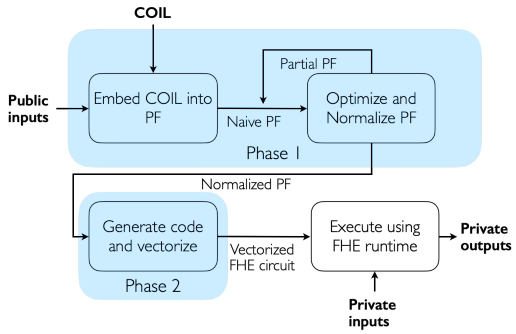


Figure 1. Coil compilation overview

later—is philosophically identical to how symbolic execution engines treat symbolic values, with one key difference: While in traditional symbolic execution, these values become concrete at run-time, FHE ciphertexts *remain unknown (to the evaluator) at runtime*. Thus, where a symbolic execution engine might discard information about explored paths after completing its analysis, for an FHE compiler *the collection of paths themselves is the executable object of interest.*

We realize this insight as a novel intermediate representation called the *path forest*, which represents all possible control flow paths through a program, with each path annotated with the sequence of conditions that must be true along it. We present a symbolic execution-like procedure for interpreting an FHE program with control flow as a path forest, and demonstrate that this representation unlocks two key optimizations: (i) *pruning* "dead" paths whose conditions are unsatisfiable; and (ii) *specializing* ciphertexts to path-dependent plaintext values when they are sufficiently constrained. Taken together, these optimizations can yield speedups of an order of magnitude over the standard muxing strategy. After pruning infeasible paths and specializing the values of `z`, Coil produces the following secure program:

```
λ(x: int, y: int).
    let [z, w] = mux(x - 1 < y, [1, 4], [2, -1]) in
    [z, w]
```

Note that specializing `z` to a plaintext does *not* leak any information to the evaluator: While the original program nominally contains four control flow paths, two of the paths are infeasible–the path forest still encodes *every feasible path.*

1.2 Coil Overview

This paper introduces Coil, a compiler that translates standard programs to ones with oblivious control flow using path forests, coupled with a runtime system that executes those path forests in an efficient, vectorized manner. At a high level, the Coil compiler proceeds in two phases, depicted in Figure 1:

1. The program is partially evaluated with respect to given plaintext inputs, specializing ciphertexts to plaintext when possible and pruning unrealizable paths, while carefully merging paths to control path explosion, yielding a path forest.
2. The path forest is lowered to vectorized primitives that can be executed using the COPSE algorithm [26] (Section 4.7).

Intuitively, our approach is secure by construction since both phases operate using *only* plaintext inputs–they cannot leak any private information. Section 4 makes this informal privacy guarantee more precise: programs written in Coil are secure-by-construction (Theorem 4.1), and the path forest compilation is semantics-preserving (Theorem 4.4).

Controlling Path Explosion. Done naïvely, the strategy outlined above can produce a number of paths exponential in the total number of branches—a phenomenon commonly referred to as *path explosion* [9, 10]. If distinguishing those paths does not yield profitable optimization opportunities, the resulting path forest can become impractically large and contain a lot of redundant work. To mitigate this exponential blowup, Coil implements a *merging* operation that identifies when unpruned paths are identical past a certain point, and folds them back together, eliminating the need to redundantly evaluate them. The language also provides a direct `mux` construct that allows the programmer to specify that a particular branch should not be split.

1.3 Contributions

The specific contributions we make in this paper are:

1. A construct called *path forests* for representing control flow in FHE programs.
2. A series of optimizations enabled by path forests, including *specialization* and *pruning*.
3. A compilation procedure from the Coil surface language to our path forest language pf.
4. An execution strategy for path forests that leverages ciphertext batching optimizations to further speed up generated code

We evaluate Coil a set of benchmarks with secure control flow taken from FHE literature. We demonstrate that Coil greatly outperforms the current standard "mux network" strategy, often automatically discovering implementations that match the structure of known-optimal approaches. In fact, the code generated by Coil's parallel execution strategy is often comparable to or substantially faster than handwritten expert implementations drawn from the literature.

2 Background

Fully Homomorphic Encryption refers to a class of encryption schemes that allow for *homomorphic computation* over ciphertexts; e.g., the sum of the encryptions of two integers is the encryption of their sum. FHE is a useful cryptographic

tool for carrying out *privacy-preserving* or *oblivious computation* (evaluating programs where some of the inputs are unknown to the evaluator).

A common use-case for FHE is *computation offloading*, which uses a single public/private keypair and only involves two parties: The client encrypts their private inputs to a program and sends the ciphertexts, along with their public inputs, to the evaluator. The evaluator executes the program over their own inputs and the client's public and private inputs via a sequence of homomorphic operations before sending the resulting ciphertext back to the client, who finally decrypts it and learns the output of the program.

Oblivious Semantics for Computation. The standard security guarantee for oblivious computation is *noninterference*: an evaluator without the private key cannot distinguish two traces of a program that differ only in encrypted variables. This poses a problem for programs with branching: the evaluator cannot know which branch to take without learning some information about the ciphertexts that influence that branch, breaking noninterference. To preserve noninterference we want programs to exhibit *oblivious control-flow semantics*, in which the evaluator correctly executes a branching program without knowing anything about which branches were taken.

A common way to provide such semantics is via *secure multiplexers* ("muxes"), cryptographic operations that use a ciphertext "selector" to obliviously choose between multiple inputs, returning the input corresponding to the value of the selector. When the evaluator encounters a branch, it executes *both* paths, and then muxes on the branching condition to select the correct value when control flow converges. This obviates the need to reveal the branching condition to the evaluator, while ensuring the correct return value. Note that this technique can, in the worst case, exponentially increase the total computation time: every path through the program has to be evaluated, despite using only one result.

Muxes can be used to implement other kinds of secure control flow: given a plaintext upper bound on the number of iterations, a loop can be fully unrolled into a series of muxes that check the exit condition at each iteration.

Armed with these constructions, we can build a compiler that achieves oblivious semantics [18, 19]: functions are inlined, loops are fully unrolled, and all the resulting branches are converted to muxes, resulting in a circuit which can be evaluated in any FHE scheme.

Ciphertext Batching. Many FHE schemes provide an optimization called *ciphertext batching*, which allows encrypting a vector of integers into a single ciphertext, such that homomorphic operations occur element-wise on the underlying vector (i.e. SIMD-style). Ciphertext batching can alleviate the FHE overhead by *vectorizing* computations which reduces the total number of homomorphic operations performed. For example, a sequence of N independent multiplies can be replaced by a single vector multiply by packing all the left and right operands into ciphertext vectors. These vector semantics have their own limitations that make vectorizing general FHE applications difficult [13, 25, 32]. While Coil does not advance ciphertext batching, the parallel execution primitives we lower to still use it as an optimization [26].

```
fn lookup(haystack: 𝒜^t, needle: int, cur: int): int =
    if cur < len(haystack) then
        if needle ≡ cur then cur
        else lookup(haystack, needle, cur + 1)
    else len(haystack)
fn index(arr: 𝒜^t, i: int, cur: int): int =
    if cur ≡ len(arr) then  -1
    else if cur ≡ i then arr[cur]
    else index(arr, i, cur + 1)
λ(keys: 𝒜^⊤, vals: 𝒜^⊤, key: int).
    let i = lookup(keys, key, 0) in
    index(vals, i, 0)
```

Figure 2. Coil snippet implementing associative array. By convention, we underline private values and operations.

Noise Management. Much of the security of FHE schemes comes from a small amount of noise added to each ciphertext upon encryption. A freshly encrypted ciphertext starts with a certain *noise budget*. When the noise level exceeds this budget, it interferes with the encrypted value, causing decryption to fail. Homomorphic multiplications accumulate noise; hence, the noise budget roughly corresponds to the *maximum multiplicative depth* of circuit that can be evaluated. The noise budget can be increased by encrypting into larger ciphertexts, which are in turn much slower to compute over. Managing the total depth is, therefore, crucial to designing efficient FHE applications. In Section 5.3, we demonstrate how the path forest strategy can reduce the multiplicative depth of a circuit.

3 A More Detailed Example

This section walks through compiling an associative array kernel using Coil, as shown in Figure 2. This example takes an associative array of `keys` and `vals` as public inputs, and the `key` to retrieve as a private input. It implements a secure `lookup` function which looks up the index of the key in the `keys` array by privately comparing it to each element, and a secure `index` function which finds an element from the `vals` array by securely comparing `i` to every possible array index,[3] returning -1 if the key is not found.

[3]Note that in the example, `lookup` returns a ciphertext, so we cannot directly index an array with the return value `i`, necessitating the use of the `index` function (Section 4.1).

3.1 Building a Path Forest

The first phase of the compiler builds a path forest by inlining function calls, substituting variables, and repeatedly applying the following rules:

- We generate paths in the forest for each ciphertext-dependent branch; e.g. for `if b then x else y` where `b` is a ciphertext, we generate the paths `(b, x)` and `(¬b, y)`.
- Whenever the value of a ciphertext variable can be determined along a particular path, we "specialize" that copy of the variable to a plaintext and substitute its value; e.g. for `if x ≡ 3 then x + 2 else x + 1`, we specialize `x` to `3` along the first branch to produce the paths `(x ≡ 3, 5)` and `(x ≠ 3, x + 1)`.
- We evaluate any plaintext-dependent branches (i.e. branches that can be determined from public inputs alone) to "prune" away unreachable paths. In particular, this includes branches that have been made plaintext-dependent by the previous step. For example:

  ```
  if x ≡ 3 then (if x ≡ 4 then x - 2 else x + 2)
  else x + 1
  ```

  since the value of `x` is known to be `3` in the first branch, we prune away the path corresponding to `x ≡ 4` and generate the same two paths as before.

As Coil steps through this procedure for the example in Figure 2, it yields a forest with a single path for each possible value of the key, as follows:

Assume compilation begins by binding the public inputs `keys ↦ [1, 2, 3]` and `vals ↦ [1, 4, 9]`. Coil begins by inlining the first call to `lookup` to produce:

```
let i = if 0 < len(keys) then (if key ≡ keys[0] then 0
                               else lookup(keys, key, 1))
        else len(keys) in
index(vals, i, 0)
```

Since the condition `0 < len(keys)` can be evaluated in plaintext, the second branch gets pruned away:

```
let i = if key ≡ keys[0] then 0 else lookup(keys, key, 1) in
index(vals, i, 0)
```

and finally, Coil simplifies the array index `keys[0]` and generates two paths from the ciphertext-dependent branch:

```
(key ≡ 1, let i = 0 in index(vals, i, 0));
(key ≠ 1, let i = lookup(keys, key, 1) in index(vals, i, 0))
```

Focusing on the first path, Coil next specializes `i ↦ 0` and inlines the call to `index`:

```
(key ≡ 1, if 0 ≡ len(vals) then -1
          else if 0 ≡ 0 then vals[0]
          else index(vals, 0, 1))
```

Note that despite `i` being a ciphertext `int`, because its value is determined to be 0 *along the first path*, Coil can prune the `-1` and `index(vals, 0, 1)` branches and specialize `val[0]` to produce `(key ≡ 1, 1)`. Continuing this procedure on the remaining paths, Coil ultimately produces the following path forest:

```
λ(keys: 𝒜⊤, vals: 𝒜⊤, key: int).
    (key ≡ 1, 1);
    (key ≠ 1 ∧ key ≡ 2, 4);
    (key ≠ 1 ∧ key ≠ 2 ∧ key ≡ 3, 9);
    (key ≠ 1 ∧ key ≠ 2 ∧ key ≠ 3, -1)
```

Note that specializing on possible values of ciphertexts does not leak any information. For instance, although `key` and `i` are ciphertexts, their values are uniquely determined on any particular path, allowing them to be replaced by plaintexts. Crucially, even though each path is evaluated using plaintext values, every path is still evaluated, and the appropriate result is selected securely.

3.2 Executing a Path Forest

Once the path forest is produced, Coil generates primitives that map to the vectorized execution model of the COPSE decision tree inference algorithm [26] by vectorizing together the conditions along each path, as well as the expressions at each leaf (Section 4.7).

Interestingly, the combination of Coil's path forest specialization and this vectorized execution often automatically generates code equivalent to well-known expert-coded implementations. For example, a well-known strategy for associative array lookup[4] involves first computing a "one-hot" indicator vector encoding the position of the key, and then computing a dot product between this vector and the values. We can compare this strategy to the one Coil generates from the implementation in Figure 2. After pruning and unraveling all the control flow paths, the remaining conditions of the form "`key ≡ keys[i]`"are essentially vectorized together, similar to the first step of the dot product strategy described above; the COPSE algorithm essentially completes the dot product computation.

Overall, given a naïve implementation of associative array lookup, Coil automatically generates an implementation akin to what a cryptographic expert might write.

4 Coil Design

This section first describes the Coil and path forest (PF) languages used by our pipeline, then explores in more detail the first compilation phase from Coil programs to PF, and finally discusses how we generate code from a path forest.

4.1 Source Language

Coil's user-facing language supports (integer) arrays with a fixed, publicly-known length, recursion, basic arithmetic operators over both *encrypted* (i.e. "private" or "ciphertext") and *unencrypted* (i.e. "public" or "plaintext") integers, and

[4] This strategy is adapted from MPC folklore [29] and is often used as a secure indexing primitive in larger protocols [7, 8]

```
e ::=                          Expressions
    | x                        variables
    | n | n                    (private) numeric literals
    | true | false             Boolean literals
    | true | false             private Boolean literals
    | e ⊕ e | e ⊕ e            (private) arithmetic operations
    | f e                      function application
    | let x = e in e           let binding
    | if b then e else e       conditional
    | mux b e e                multiplexer
    | [ē]                      array
    | e[e]                     array lookup
    | e[e ⟶ e]                 array update

b ::=                          Conditions
    | e ≡ e | e ≡ e            (private) equality
    | e < e | e ≤ e            (private) inequality
    | ¬e | ¬e                  (private) negation
    ℓ ::= ⊤ | ⊥                    Security Labels

    D ::= fn f(x̄ : τ̄):τ = e    Function Definitions

    p ::= λ(x̄ : τ̄). [ē]        Coil Program
```

Figure 3. Syntax of the Coil surface language.

```
c ::=                          Path Clauses
    | ne ≡ ne                  private equality
    | ne ≤ ne                  private inequality
    | ¬c                       private negation
p ::=c̄                         Paths
r ::=                          Path Forests
    | λ(x̄ : τ̄). [p̄, ē]
ne ::=                         FHE Expressions
    | x                        private variables
    | n                        private literals
    | ne ⊕ ne                  private arithmetic ops
    | mux b ne ne              muxes
nr ::=                         Normal Path Forests
    | λ(x̄ : τ̄). [p̄, n̄e]
```

$$\frac{p_i \longrightarrow^* \texttt{true}\ \ e_i[\bar{x} \mapsto \bar{v}] \longrightarrow^* v}{(\lambda(\bar{x} : \bar{\tau}).\ (p_1, e_1);\ldots;(p_k, e_k))(\bar{v}) \longrightarrow^* v}\ \text{Ev-Pf}$$

$$\frac{\forall\ i.\ \bar{x}: \bar{\tau} \vdash c_i: \texttt{bool} \quad \forall\ i.\ \bar{x}: \bar{\tau} \vdash e_i: \tau}{\vdash \lambda(\bar{x} : \bar{\tau}).\ (c_1, e_1);\ldots;(c_k, e_k): \bar{\tau} \to \tau}\ \text{T-Pf}$$

Figure 4. Syntax, semantics, and typing rules for Pf. $\bar{e}$, $\bar{p}$ represent lists of expressions, paths, etc.

conditional expressions with both public and private conditions. To formalize our compilation strategy, we will use a fragment of Taype, a privacy-preserving programming language developed in prior work [35]. The grammar of this fragment is shown in Figure 3 and it directly adopts the semantics and security type system of Taype as its own.

In addition to all the features mentioned above, Taype provides a set of private operations that ensure that programs do not leak information through control flow channels. The multiplexer operation `mux`, for example, is a secure conditional that fully evaluates both of its branches before producing a result, thus ensuring it always generates the same execution trace regardless of the private condition. As a consequence, replacing the $\underline{3}$ in the following expression yields an identical execution trace modulo the private values that appear:

```
mux (3 ≤ 4) (5 + 1) (6 + 1) ⟶ mux true (5 + 1) (6 +1)
   ⟶ mux true 6 (6 + 1) ⟶ mux true 6 7 ⟶ 6
```

Taype is also equipped with secure variants of the standard arithmetic operations, e.g., $\underline{+}$, $\underline{\leq}$. Importantly, users do not have to explicitly include any secure operations in a program—instead, they can write their program using standard arithmetic operations and `if`, and Taype's *tape semantics* [34] will automatically insert their secure variants at runtime. However, the language does provide an explicit `mux` construct that gives programmers fine-grained control over the shape of the compiled path forest (see Section 6.1).

To fit the encrypted computation paradigm, well-behaved Coil programs must satisfy the following restrictions:

- All array indices must be publicly known[5] (this is enforced by Coil's type system).
- Recursive functions must terminate *based on publicly known inputs*, since otherwise, evaluating a recursive function would leak something about its termination condition (Section 4.5).

In the example program in Figure 2, the first restriction means that since `cur` is used as an array index in both `index` and `lookup`, it must be a plaintext input. The example satisfies the second restriction because, for instance, the recursive calls to `index` can be inlined until `cur ≡ len(array)`, a condition that involves only plaintext values. Thanks to its tape semantics, all well-typed Taype (and thus Coil) programs enjoy the following obliviousness theorem:

Theorem 4.1 (Obliviousness [35]). *If* $e_1 \approx e_2$ *and* $\cdot \vdash e_1 :_{l_1} \tau_1$ *and* $\cdot \vdash e_2 :_{l_2} \tau_2$, *then*

1. $e_1 \longrightarrow^n e_1'$ *if and only if* $e_2 \longrightarrow^n e_2'$ *for some* e_2'.
2. *if* $e_1 \longrightarrow^n e_1'$ *and* $e_2 \longrightarrow^n e_2'$, *then* $e_1' \approx e_2'$.

Here, $e_1 \approx e_2$ means the two expressions are *indistinguishable*, i.e. they only differ in their private inputs, and $e \longrightarrow^n e'$ means e reduces to e' in exactly n steps. A direct corollary of this theorem is that when a well-typed Coil program is supplied with the same public inputs (but not necessarily the same private ones), it will produce execution traces that appear equivalent to the evaluator.

4.2 Path Forests

Our path forest language PF is a straightforward extension of Coil; its additions are shown in Figure 4. A PF *path forest* consists of a sequence of Coil expressions, each of which is labeled by a *path*, or conjunction of Boolean conditions (*path clauses*). Intuitively, each of the expressions are different versions of the same Coil program specialized along each given path. Thus, the path forest can be thought of as explicitly encoding possible control-flow paths through a Coil program. PF inherits most of its semantics directly from Coil, only adding the Ev-Pf evaluation rule, which allows us to step the path forest to an expression if its corresponding path evaluates to `true` given a set of ciphertext inputs $\overline{v}$. Its sole additional typing rule, T-Pf states that a path-forest is well-typed if each of the paths is a Boolean, and each of the expressions has the same type.

The structure of the Ev-Pf rule immediately motivates the following definition:

Definition 4.2 (Well-Formedness)**.** A path forest `r = λ(x̄: τ̄). (p_i, e_i);...` is *well-formed* (written $\Gamma \vdash \texttt{WF(r)}$) if $\Gamma \vdash \texttt{r} : \tau$, and for all ciphertext inputs $\overline{v}$, $\exists i\ \texttt{p}_i(\overline{v}) \longrightarrow^* \texttt{true}$ and $\forall\ \texttt{j} \neq \texttt{i}.\ \texttt{p}_j(\overline{v}) \longrightarrow^* \texttt{false}$

In other words, a well-formed path forest (i) can be deterministically reduced to a single value; and (ii) as a consequence of Theorem 4.1, is secure by construction.

Note that every Coil program can be lifted to PF by creating a single path with a condition of `true`. Since a "normal-form" path forest (ne in Figure 4) is exactly one which doesn't contain any conditionals—in other words, one in which all branching is captured either by explicit muxes, or in the path conditions—the goal of compilation is to convert a Coil program into a normal-form path forest, as described below.

4.3 Compilation

Compilation begins only once plaintext inputs are available.[6] We embed `λ(x̄: τ̄). e` into PF as `λ(x̄: τ̄). (true, e)`, partially apply it with respect to the plaintext inputs, and then proceed according to the $\cdot \rightsquigarrow \cdot$ rule presented in Figure 5 until the path forest is in a normal form. In other words, we repeatedly apply "$\rightsquigarrow$" to *partially evaluate* each expression in the path forest with respect to both (1) information knowable from its corresponding path clauses (*pruning*), and (2) values known in plaintext (*specialization*), and generate new paths with appropriate path clauses whenever we encounter branching control flow. A normal-form path forest is then exactly one

[5]Oblivious lookups are possible by, for example, writing a function such as the one described in Section 3.2. The design of the language allows this to happen with zero abstraction overhead, as discussed later in this section.

[6]This means that Coil programs can be compiled ahead of time if the plaintext inputs are unlikely to change, and otherwise must be compiled just-in-time. The timings presented in Section 5 are broken down to include both compilation and run time.

$$\frac{\overline{x}: \overline{\tau} \vdash e : \texttt{int} \quad e \longrightarrow^* n}{\lambda(\overline{x}: \overline{\tau}).\ (p, e) \rightsquigarrow \lambda(\overline{x}: \overline{\tau}).\ (p, n)}\ \textsc{Pf-Eval}$$

$$\frac{p \implies b}{(p, \texttt{if b then } e_1 \texttt{ else } e_2) \rightsquigarrow (p, e_1)}\ \textsc{Pf-IfT}$$

$$\frac{p \implies \neg b}{(p, \texttt{if b then } e_1 \texttt{ else } e_2) \rightsquigarrow (p, e_2)}\ \textsc{Pf-IfF}$$

$$\frac{}{(p, \texttt{if b then } e_1 \texttt{ else } e_2) \rightsquigarrow (p \wedge b, e_1);\ (p \wedge \neg b, e_2)}\ \textsc{Pf-IfB}$$

$$\frac{(p, e_1) \rightsquigarrow (p_i', e_i');\ldots}{(p, \texttt{let x = } e_1 \texttt{ in } e_2) \rightsquigarrow (p_i', \texttt{let x = } e_i' \texttt{ in } e_2)}\ \textsc{Pf-Let1}$$

$$\frac{}{(p, \texttt{let x = ne in e}) \rightsquigarrow (p, \texttt{e}[x \mapsto \texttt{ne}])}\ \textsc{Pf-Let2}$$

Figure 5. Selected evaluation rules for PF. The rules are implemented in a judgement $\cdot \rightsquigarrow \cdot$. The metavariables e and ne refer to arbitrary Coil expressions and normal form Coil expressions, respectively. `λ(x̄: τ̄)` is omitted for clarity.

which cannot be evaluated any further based on plaintext information. More precisely:

Theorem 4.3 (Progress)**.** *If* $\Gamma \vdash \texttt{r} : \tau$ *is a well-typed path forest, then either* r *is in path forest normal form or there is some path forest* r′ *such that* $\texttt{r} \rightsquigarrow \texttt{r}'$.

Of particular note in Figure 5 are the rules for `let`-expressions, `if`-statements, and arithmetic. The `let` rules implement *substitution*: when we encounter an expression of the form `let x = e1 in e2`, we generate a path forest with paths that replace each x in e_2 with every possible path resulting from stepping e_1. The Pf-IfB rule generates two paths: one that computes e_1 under the condition b, and one that computes e_2 under the condition ¬b.

Finally, given an arithmetic operation like $e_1 \oplus e_2$, Coil first produces paths for both e_1 and e_2, and then generates a path for the operation for every pair of operand paths. This procedure has a multiplicative effect on the number of paths in the forest, and thus, the final number of paths generated by $\rightsquigarrow$ is roughly *exponential* in the number of ciphertext-dependent branches in the original Coil program. In practice, we find that many of these branches are correlated enough to be *pruned* (Section 4.4.1). However, when this is not the case, we employ the "merging" strategy described in Section 4.6. Other strategies for dealing with the exponential number of paths are discussed in Section 6.

Example. Consider the following Coil program embedded in PF:

```
λ(x: int, y: int, z: int).
  (true, z + let w = if x + y ≤ y then y else z in 2 * w)
```

We first apply the Pf-Let rule to substitute *w* and step to:

```
λ(x: int, y: int, z: int).
  (true, z + 2 * (if x + y ≤ y then y else z))
```

Since the only path (true) cannot prove anything about the relationship between x and y, we take the Pf-IfB rule and generate the following two paths:

```
λ(x: int, y: int, z: int).
  (x + y < y, z ± 2 * y);
  (x + y ≥ y, z ± 2 * z)
```

The path forest is now in normal form and there are no more rules to apply.

The following soundness theorem ensures that $\cdot \rightsquigarrow^* \cdot$ preserves the semantics of the original Coil program:

Theorem 4.4 (Soundness). *Let* `r = λ(x̄: τ̄). p` *be a path forest such that* $\cdot \vdash$ `WF(r)`*, and let* $\bar{v}$ *be a set of ciphertext inputs. Then if* $r(\bar{v}) \longrightarrow^* v$ *and* $r \rightsquigarrow^* r'$, $r'(\bar{v}) \longrightarrow^* v$.

4.4 Optimizations on Path Forests

Recall from Section 3 that the motivating principle behind introducing the pf language is that *path forests enable path-dependent optimization*. We now describe how the evaluation rules presented in Figure 5 recover our two main optimizations: *pruning* unreachable paths, and *specializing* to known (plaintext) values.

4.4.1 Pruning. Consider again the previous example, but this time let x have plaintext type int instead of ciphertext type int, and suppose compilation starts by partially applying $x \mapsto 3$. Then once we've stepped to the forest:

```
λ(x: int, y: int, z: int).
  (true, z ± 2 * (if x ± y ≤ y then y else z))
```

we can prove $\texttt{true} \implies \neg(\texttt{3 + y < y})$, allowing us to apply the Pf-IfF rule instead of Pf-IfB, and stepping to:

```
λ(x: int, y: int, z: int). (true, z + 2 * z)
```

In practice, when we encounter an expression of the form `(p, if b then e₁ else e₂)`, we generate the queries $p \wedge b$ and $p \wedge \neg b$ and discharge them to a solver.[7] If one of the queries is unsatisfiable we take the corresponding Pf-IfT or Pf-IfF rule; otherwise, we take the Pf-IfB rule. Note that this means that paths are pruned if they can be *proven unreachable based on information available at plaintext time*, even if the condition itself is not plaintext: for example, knowing the value of x is enough to decide whether or not $x + y < y$, even though both $x + y$ and y have private type int!

We can understand the power of this by looking at a slightly more complicated example:

```
λ(x: int, y: int, z: int).
  let w = (if x ≤ 0 then y else z) in
  if x ± y ≤ y then z * w else z ± w
```

We start by taking the Pf-Let and Pf-IfB rules and arrive at:

```
fn midpt(x: int, y: int): int =
  if (y - x < 2) then x
  else midpt(x + 1, y - 1)
fn search(arr: 𝒜^ℓ, key: int, lo: int, hi: int): int =
  let mid = midpt(lo, hi) in
  if arr[lo] ≡ key then ↑lo else
    if arr[lo] ≤ key then search(arr, key, mid, hi)
    else search(arr, key, lo, mid)
λ(arr: 𝒜^ℓ, key: int, len: int). search(arr, key, 0, len)
```

Figure 6. Snippet of a Coil program implementing a binary search over an array of encrypted data. While the language does not natively support division, the programmer can implement a `midpt` function over plaintext (ints) without incurring a run-time overhead.

```
λ(x: int, y: int, z: int).
  (x + y < y, z * (if x ≤ 0 then y else z));
  (x + y ≥ y, z ± (if x ≤ 0 then y else z))
```

Now, stepping the first path, we see that despite x, y, z all being ciphertexts, $\texttt{x + y < y} \implies \texttt{x < 0}$; taking the Pf-IfT rule instead of Pf-IfB (and, dually, Pf-IfF on the second path), we arrive at:

```
λ(x: int, y: int, z: int).
  (x + y < y, z * y); (x + y ≥ y, z ± z)
```

Notably, the final forest has only two paths, compared to the four that would be generated by naïvely unfolding conditionals: The Pf-IfT and Pf-IfF rules prune the two unreachable paths.

4.4.2 Specialization. Let us consider the original example one more time, but now let *both x and y* have plaintext type int, and start compilation by partially applying $x \mapsto 3$ and $y \mapsto 4$. We once again apply the Pf-Let and Pf-IfF rules to arrive at:

```
λ(x: int, y: int, z: int). (true, z ± 2 * x)
```

Since `2 * x : int` this path forest is *not* in normal form. Using $\texttt{2 * x} \longrightarrow^* \texttt{6}$, we can apply the Pf-Eval rule and step to:

```
λ(x: int, y: int, z: int). (true, z ± 6)
```

This optimization has a few consequences. Of course, it reduces the amount of computation that needs to happen when we finally generate FHE code. Furthermore, since array indices are required to be plaintext, they can necessarily be computed during path forest conversion and compiled away; thus, the language can operate over arrays without needing the arrays to show up in the final ciphertex computation. Finally, it enables programming with *zero-cost abstractions*. For example, consider the snippet of a binary search implementation in Figure 6 in which the programmer writes a function that loops to compute the midpoint of two indices.[8] Since

[7] Coil discharges these queries via a lightweight custom solver for linear arithmetic and inequalities. A full-featured SMT solver could potentially prune more paths at the cost of longer compile times.

[8] Writing a function to calculate midpoints is necessary because most FHE schemes do not natively support integer division, and hence Coil does not provide a division operator.

the `midpt` function returns an `int`, it can be executed entirely by $\rightsquigarrow$, obviating the need to generate code that executes the expensive loop over ciphertext inputs later.

4.5 Recursive Functions

In addition to producing smaller path forests (and therefore more efficient ciphertext computation), the optimizations described above also enable Coil to gracefully compile programs with recursive functions without requiring any special care. Note that a strategy that naïvely replaces conditionals with `mux`es fails to terminate in the presence of recursive functions; In particular, the original Pf-IfB rule always generates two paths, which means it *always generates a path with the recursive case*, and hence applying $\cdot \rightsquigarrow \cdot$ never terminates! In contrast, if at some point during compilation Coil can prove that the recursive branch does not get taken, then by taking either the Pf-IfT or Pf-IfF rule, we generate *only the path for the base case*. Note that the second condition in Section 4.1 (that recursion termination conditions depend only on plaintexts) guarantees that at some point during compilation, the recursive path will be pruned.

4.6 Merging Paths

Recall that in the absence of pruneable paths, the rules described in Section 4.3 produce a forest with size exponential in the number of branches in the original program. In this section, we describe a transformation that takes a path forest in normal-form and "merges together" adjacent paths that contain repeated computations, thus reducing the total number of paths.

We begin by adding a new transformation that replaces a pair of paths $(\mathtt{p}_1, \mathtt{e}_1); (\mathtt{p}_2, \mathtt{e}_2)$ with a single path of the form $(\mathtt{p}_1 \vee \mathtt{p}_2, [\![\mathtt{p}_1 \ ? \ \mathtt{e}_1 : \mathtt{e}_2]\!])$ whenever $\mathtt{e}_1$ and $\mathtt{e}_2$ are ciphertexts. At a high level, $[\![\cdot \ ? \ \cdot : \cdot]\!]$ behaves as a "lazy mux" that attempts to pull out as much repeated computation as possible between $\mathtt{e}_1$ and $\mathtt{e}_2$ before branching on the path conditions in $\mathtt{p}_1$. More precisely, a lazy mux of two arithmetic expressions which share an operand can be pushed into the non-common operand (e.g., `[[b ? x + y : x + z]]` becomes `x + [[b ? y : z]]`); a lazy mux of two arrays that differ in only one element can be pushed into the differing element (e.g. `[[b ? [x, z] : [x, w] ]]` becomes `[x, [[b ? z : w]] ]`); and a lazy mux that cannot be pushed any further can be replaced with an ordinary multiplexer (e.g. `[[b ? x : y]]` becomes `mux(b, x, y)`)[9].

The definition of $[\![\cdot \ ? \ \cdot : \cdot]\!]$ is intentionally conservative:

- We don't push $[\![\cdot \ ? \ \cdot : \cdot]\!]$ through arrays that differ in multiple positions, or arithmetic expressions that don't share an operand. While this may preclude some optimization opportunities, it guarantees that we don't generate any extra muxes.
- The rules only match on *syntactic equality* equality, rather than *logical equivalence* (e.g., the expression `[[b ? x + y : y + x]]` cannot be simplified to `x + y` even though `+` is commutative). Syntactic equality is usually sufficient, since whenever $\cdot \rightsquigarrow \cdot$ results in repeated work, it creates syntactically identical expressions.

Being conservative allows us to avoid incorporating heavyweight techniques like equality saturation and appealing to sophisticated cost models. The rules as written can never degrade performance by introducing additional muxes., and thus cannot degrade performance.

In practice, Coil fully generates a normal form path forest, and then eagerly merges the resulting paths whenever possible. We could in principle merge paths at any point during compilation, reducing the total number of paths that need to be processed at the cost of some optimization opportunities.

4.7 Generating Code

The second phase of compilation takes the optimized path forest produced as above, and uses it to generate parallelized FHE code that operates on the path forest's ciphertext inputs. The code generation strategy adapts the parallel *decision tree* algorithm described in COPSE [26] to deal with the more general structure of path forests. COPSE encodes the structure of a decision forest as a sequence of Boolean matrices $(A_{ij})_k$ together with an auxiliary *decision vector* d_i, where position (i, j) in the k-th matrix is set to 1 if the k-th branching condition along the j-th path through the forest corresponds to the element d_i in the decision vector, and 0 otherwise.

We extend this encoding to apply to path forests by collecting all the path conditions in the forest into a vector d_i and building $(A_{ij})_k$ similarly. Running this extended COPSE algorithm then produces a one-hot ciphertext vector indicating which path should be taken[10]; We use an off-the-shelf FHE vectorizer [25] to vectorize all the path expressions, and finally take the dot product of this vector with the COPSE output to return the program result.

5 Implementation & Evaluation

Coil is implemented in roughly two thousand lines of Python. We target the HElib library as our backend [20] and use the mod-2 variant of the BFV/BGV scheme. To determine the effectiveness of the compilation techniques presented in this paper, we ask the following research questions:

- **RQ1:** How do the programs that Coil generates compare to *other compilation strategies*?
- **RQ2:** How does Coil compare to *bespoke custom protocols*?
- **RQ3:** What are the *reasons* for Coil's speedups?

[9]The semantics of $[\![\cdot \ ? \ \cdot : \cdot]\!]$ are described formally in the supplemental material.

[10]Although path forests are not trees, the resulting ciphertext vector is guaranteed to have exactly one 1 by Definition 4.2.

All experiments are run on 2020 M1 MacBook Air with 16GB of RAM; the values reported are the medians across eleven runs and a 95% confidence interval.

5.1 RQ1: How efficient are the programs Coil generates?

There is no standard set of FHE benchmarks, and especially no set that makes use of oblivious control flow. Thus, to answer this question, we compile the following set of benchmarks:

1. `linear-index`: looking up a *ciphertext* index into an array of 16 elements via a linear scan
2. `log-index`: looking up a *ciphertext* index into an array of 16 elements via a binary search
3. `associative-array`: using a secure key to look up a value in an associative array of 8 elements
4. `filter`: using a threshold predicate to filter a list of 8 elements
5. `merge`: merging two sorted 5-element arrays into a sorted array of 10 elements
6. `sp-auction`: determining the winning bidder and bid in a second-price auction with 8 bidders

We compare the run times of the Coil-compiled versions against secure implementations that naïvely replace an `if` with a `mux` whenever its condition uses a ciphertext.[11] Each benchmark is implemented in Coil's surface language, and compiled down to calls to HElib. Full implementations of each benchmark are included in the supplementary material.

The compilation times for each benchmark are reported in Table 1, broken down by phase. Notice that with the exception of `merge` and `filter`, the compilation time is negligible (on the order of a few hundred milliseconds). The high compilation times can be attributed to the combinatorial explosion of paths, a phenomenon which is discussed in more detail in Section 5.3, although it is worth noting that in both cases the re-folding mitigates the path explosion reasonably well.

The times and speedups for each benchmark are reported in Table 1. We see that Coil outperforms naïve implementations, sometimes by as much as two orders of magnitude. These speedups are most prominent on the benchmarks containing instances of an array being indexed by a ciphertext, as Coil's unique specialization and pruning strategy is able to avoid the overhead of these expensive indexing operations. In fact, the data for the naïve implementation of the `merge` benchmark is missing: even on modest input sizes (e.g. merging two arrays of size 5) it times out after 30 minutes.

[11] Our naïve implementations are manually transliterated from the Coil surface language to C++. An example of this translation is shown in Figure 7a.

5.2 RQ2: How does Coil compare to known custom protocols?

With some of our benchmarks, a naïve translation of a Coil implementation is unfair, as we already know of efficient specialized protocols that implement them. In particular, we can use the one-hot vector and dot product strategy described in Section 3.2 for `linear-index`, `log-index`, and `associative-array`, a single vectorized comparison and mux for `filter`, and part of a custom k-way sorting protocol for `merge`[22].

Table 1 also shows a comparison of Coil's execution time to each of these custom protocols. With the exception of `filter`, the Coil version *outperforms* its existing custom protocol. Even for `filter`, although the path unfolding *does* produce an exponential number of paths, all of these end up refolded together, and the final generated code is nearly identical to the expert implementation.

The other benchmarks see significant speedups. For secure array indexing and associate array lookup, Coil matches the dot product technique, and its vectorized execution strategy provides further speedup. Explaining the speedup for `merge` is more interesting, as the next section elaborates.

5.3 RQ3: Where do Coil's speedups come from?

We first describe the challenges associated with implementing the `merge` benchmark, and analyze how Coil's compilation strategy enables optimization opportunities that address these challenges. Figure 7 shows two partial implementations of a function that merges sorted arrays, one in Coil's surface language (Figure 7b) and one that naïvely translates it to C++ (Figure 7a). We analyze the naïve implementation, and contrast it to Coil's evaluation strategy.

What does a naïve merge look like? Each iteration of the for loop contains (1) two secure array index operations, (2) a secure comparison, and (3) two oblivious updates to the array indices. Each of the multiple inefficient secure array lookups contributes significantly to the overall multiplicative depth. Furthermore, the entire procedure is inherently sequential, making it difficult to recover performance via parallelization or vectorization. Finally, since each iteration uses the ciphertext index values updated from the previous iteration, the total multiplicative depth *stacks*, resulting in circuits that either require huge parameters to evaluate or multiple expensive rounds of bootstrapping; Indeed, our naïve implementation of `merge` times out after running for 30 minutes (Table 1).

What does Coil do differently? The Coil execution strategy transforms the snippet in Figure 7b into something much more efficient. Each specialization and pruning step inlines one level of the recursive `merge` call, eventually unfolding the entire program into a large binary tree representing *every possible way to merge the lists*, where every node corresponds

Table 1. Results of Coil on our benchmarks. The first set of columns reports compile times, broken down by phase. The second group reports speedups of Coil-compiled programs over naïve and expert implementations. Note that some numbers are missing: the naïve merge times out, and we do not have an expert implementation available for second-price auction.

Benchmark	Compilation Time (ms)			Coil	Naïve		Expert	
	Phase 1	Phase 2	Total	Time (s)	Time (s)	Speedup	Time (s)	Speedup
linear-index	25	290	315	2.92	67.66	23.16×	5.06	1.73×
log-index	48	263	311	1.48	36.34	24.55×	5.06	3.42×
assoc-array	24	277	301	1.59	18.67	11.75×	5.66	3.56×
filter	31	1539	1570	1.01	17.9	17.76×	0.85	0.85×
merge	120	6730	6850	11.55	—	—	51.32	4.44×
sp-auction	57	370	427	15.4	833.93	54.13×	—	—

```
ctxt i1, i2 = encrypt(0);
for (int i = 0; i < len(arr1) + len(arr2);
     i++)
{
  ctxt arr1Val = index(arr1, i1);
  ctxt arr2Val = index(arr2, i2);
  ctxt b = compare(arr1Val, arr2Val);
  output[i++] = mux(b, arr1Val, arr2Val);
  i1 = mux(b, i1 + 1, i1);
  i2 = mux(b, i2, i2 + 1);
}
```

(a) Naïve implementation of merge in C++

```
fn merge(arr1: A^ℓ, arr2: A^ℓ, out: A^ℓ,
         i1: int, i2: int, j: int): A^ℓ =
  ...
  if arr1[i1] < arr2[i2] then
    merge(arr1, arr2, out[ j → arr1[i1] ],
          i1 + 1, i2, j)
  else
    merge(arr1, arr2, out[ j → arr2[i2] ],
          i1, i2 + 1, j)
  ...
```

(b) Implementation of merge in Coil

Figure 7. Example snippets merging two sorted arrays of ciphertexts in C++ and in the Coil surface language. Note that an actual implementation of `merge` would include bounds checks for the two indices `i1` and `i2`; these checks have been omitted from the above code for the sake of clarity.

Table 2. Breakdown of time spent evaluating path conditions vs. expressions in the generated code

Benchmark	Branches (ms)	Labels (ms)
linear-index	2840	5
log-index	1710	4
sp-auction	6930	9530
merge	12400	30
filter	0	920
associative-array	2070	3

to a possible pair of histories for `i1` and `i2`. Finally, Coil generates code to choose which merged array to select.

This representation has a number of effects: first, since all the array indices can be specialized into plaintext values, we obviate the need for the expensive secure index operations that the naïve implementation uses; in particular, this greatly reduces the multiplicative depth of the overall circuit, allowing it to be evaluated with smaller parameters. Second, by separating out all the paths through the function we greatly increase the amount of parallelism available: every array comparison can be evaluated in parallel, and selecting the correct path at the end can be done via efficient parallelized and vectorized operations [26]. Taken together, these exactly solve the issues with the naïve implementation.

Notice that there are an exponential number of ways to merge two lists, and hence an exponential number of paths to evaluate. Certainly, separating out all the control flow paths *seems* like a very bad idea: the amount of work increases from $O\left(n^2\right)$ to something like $O\left(4^n\right)$. However, we also go from having to evaluate everything sequentially to having multiple degrees of parallelism, as well as being able to exploit ciphertext batching. Of course, at some point the size of the arrays grows beyond what FHE vectors can reasonably hold, and we need a new approach. Indeed, there are several optimizations that can be performed on top of the version of the Coil strategy presented here to allow it to scale to significantly larger programs (Section 6).

5.4 Discussion

Recall that one of the advantages of representing a program as a path forest is the ability to then convert it to vectorized execution primitives such as COPSE [26]. Because it is difficult to disentangle conditionals from other computation in the baseline code, we cannot directly measure the speedup we get from being able to perform this conversion. However, we can estimate it as follows: The cost of evaluating the conditionals can reasonably be bounded by the cost of sequentially evaluating the path forest that Coil generates. While we do not have access to an efficient sequential decision tree implementation, prior work shows that the COPSE

algorithm yields a speedup of $\sim 5\times$ [26]; Thus, we expect the performance impact of our extended COPSE implementation to be similar. As Table 2 shows, even after this speedup, for most of our benchmarks the vast majority of the execution time is spent evaluating the path conditions themselves.[12] We can therefore estimate the benefit of decision tree conversion plus efficient inference as approximately 5×. That leaves the additional 2×-10× speedup over the baseline attributable to the specialization and pruning enabled by path forests.

6 Scalability

In this section we briefly describe possible ways to deal with the path explosion that can result from applying the path forest strategy to certain programs, as well as how our techniques compare to classical path-sensitive optimizations.

6.1 Path Explosion

The merging rule from Section 4.6 helps control the number of paths in the generated code. However, the compiler still has to visit the exponentially many paths produced by $\cdot \rightsquigarrow \cdot$. Thus, even if all spurious paths can be merged, the path explosion problem still shows up at *compile time*. This is unsurprising: any compiler that aggressively tries to use path-dependent information will have to deal with path explosion.

Linearization. One might consider a strategy that *entirely avoids* unfolding branches that do not result in a benefit from pruning and specialization, and hence are not "worth it." In fact, this corresponds to a well-known technique called *control flow linearization*, in which explicitly branching control flow is replaced with a branching dataflow operation like a `mux`. [4, 5, 17, 24, 27] Manually performing linearization is already possible: A programmer who notices that a particular branch is not worth unfolding can replace that branch with a `mux` in the original Coil program. A `mux` is just another branchless computation, so it does not generate an extra path in the forest, and hence does not contribute to path explosion. Performing linearization *automatically* is also, in principle, possible: The compiler can choose to merge paths *while generating them* instead of waiting until the end. Unfortunately, in practice this is a bit more subtle: Merging too eagerly during compilation precludes applying any path-dependent optimizations that might use the corresponding branching condition. Knowing in advance which branches will provide information useful for optimization is incredibly difficult since the corresponding optimization point may be arbitrarily deep in the program; thus, our current implementation of Coil does not employ this technique. However, we believe that developing this analysis is an interesting future research direction to pursue.

[12]The exceptions are `filter` and `sp-auction`. For `filter`, the re-folding pass figures out that the optimal strategy involves just directly muxing, and thus there are no branches in the resulting decision tree; for `sp-auction` only some of the branches get folded into muxes.

6.2 Blocking

While Coil can amortize much of the exponential blowup via vectorization and parallelism, FHE vector widths are not infinite. This restriction is particularly relevant to the COPSE algorithm, which treats decision tree evaluation as a series of matrix multiplications, and exploits ciphertext batching by packing matrices into ciphertext vectors large enough to hold them. A common workaround is *blocking*: If a particular set of FHE parameters allows for vectors capable of operating on 1000×1000 matrices, we can operate on an 8000×8000 matrix simply by blocking it into 64 submatrices and then operating on each submatrix. Of course, blocking gives up the asymptotic benefits of vectorization as the demand for vector lanes increases. However, FHE vectors are incredibly wide, still allowing for significant speedups over the alternative.

7 Related Work

Compiling Oblivious Control Flow. A number of FHE compilers [2, 6, 11, 13, 14, 16, 18, 25, 32] provide a high-level surface language to write programs in, and then compile these programs down into circuits that can be executed within FHE. The TFHE Transpiler and HEIR [2, 18] toolchains are notably different from the other compilers listed above in that they can *handle oblivious control flow*: they use a Boolean circuit-based backend, so non-polynomial comparison operators are straightforward to implement, and conditionals can be emulated via muxes. The particular FHE backend that the Google Transpiler uses is TFHE [12]. While TFHE allows for incredibly fast bootstrapping and hence lends itself well to efficient implementations of large Boolean circuits, it does *not* support ciphertext batching (Section 2), making it unsuitable for our vectorized code generation strategy. Furthermore, both HEIR and the Transpiler require the programmer to express control flow as multiplexed circuits; these correspond to our manual baseline in Section 5.

Reducing Control Flow. The problem of control flow is not unique to FHE. For programs running on GPUs, branching control flow can mean some threads sit idle for parts of the program, resulting in low utilization and poor performance. Various techniques have been proposed to deal with this problem [4, 5, 17, 24, 27, 28, 30, 31]. Many of these techniques often either rely on instruction set features missing from most FHE backends, or end up doing something semantically equivalent to muxes anyway. However, as we noted in Section 6, applying some control flow reduction techniques to programs before using Coil can help further improve performance.

Other Path-Sensitive Analyses. Coil's optimizations can be thought of path-sensitive variants of traditional techniques: *specialization* is a path-sensitive version of constant propagation, *pruning* effectively identifies infeasible paths to perform dead code elimination, and the evaluation rules

of ⇝ are similar to *partially evaluating* the input program with respect to plaintext values [3, 15, 21, 23, 33]. Coil differs from these traditional techniques in two key ways. First, Coil is much more aggressive than traditional path-sensitive dataflow analyses. Specializing with respect to *plaintext values* instead of just *compile-time constants* means more path-dependent information is available, and thus more computation can be specialized, and more paths can be pruned. Second, rather than treating the control-flow structure of the program as a given, Coil explicitly unfolds branches to *expose more paths*. While this is normally not a good idea because of the compile-time impact, ciphertext computations are so expensive that the benefits of being able to eliminate some make this tradeoff worth it.

8 Conclusion

This paper presents Coil, a language and evaluation strategy for FHE programming with secure control flow. We show how Coil can transform programs with secure control flow into path forests, and present a strategy that executes these path forests via efficient decision tree evaluation protocols. We demonstrate on a variety of benchmarks that Coil outperforms other state-of-the-art techniques by up to an order of magnitude. Coil thus represents the next major step towards making FHE programs more efficient.

Acknowledgements

No generative AI tools were used in preparing this draft.

References

[1] Coşku Acay, Rolph Recto, Joshua Gancher, Andrew C. Myers, and Elaine Shi. 2021. Viaduct: An Extensible, Optimizing Compiler for Secure Distributed Programs. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) *(PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 740–755. doi:10.1145/3453483.3454074

[2] Asra Ali, Jaeho Choi, Bryant Gipson, Shruthi Gorantala, Jeremy Kun, Wouter Legiest, Lawrence Lim, Alexander Viand, Meron Zerihun Demissie, and Hongren Zheng. 2025. HEIR: A Universal Compiler for Homomorphic Encryption. arXiv:2508.11095 [cs.CR] https://arxiv.org/abs/2508.11095

[3] F. E. Allen and J. Cocke. 1976. A program data flow analysis procedure. *Commun. ACM* 19, 3 (March 1976), 137. doi:10.1145/360018.360025

[4] J. R. Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. 1983. Conversion of Control Dependence to Data Dependence. In *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Austin, Texas) *(POPL '83)*. Association for Computing Machinery, New York, NY, USA, 177–189. doi:10.1145/567067.567085

[5] Jayvant Anantpur and Govindarajan R. 2014. Taming Control Divergence in GPUs through Control Flow Linearization. In *Compiler Construction*, Albert Cohen (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 133–153.

[6] David W. Archer, José Manuel Calderón Trilla, Jason Dagit, Alex Malozemoff, Yuriy Polyakov, Kurt Rohloff, and Gerard Ryan. 2019. RAMPARTS: A Programmer-Friendly System for Building Homomorphic Encryption Applications. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography* (London, United Kingdom) *(WAHC'19)*. Association for Computing Machinery, New York, NY, USA, 57–68. doi:10.1145/3338469.3358945

[7] Fattaneh Bayatbabolghani, Marina Blanton, Mehrdad Aliasgari, and Michael T. Goodrich. 2017. Secure Fingerprint Alignment and Matching Protocols. *CoRR* abs/1702.03379 (2017). arXiv:1702.03379 http://arxiv.org/abs/1702.03379

[8] Marina Blanton, Ahreum Kang, and Chen Yuan. 2019. Improved Building Blocks for Secure Multi-Party Computation based on Secret Sharing with Honest Majority. Cryptology ePrint Archive, Paper 2019/718. https://eprint.iacr.org/2019/718. https://eprint.iacr.org/2019/718

[9] Peter Boonstoppel, Cristian Cadar, and Dawson Engler. 2008. RWset: Attacking Path Explosion in Constraint-Based Test Generation. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 351–366.

[10] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) *(OSDI'08)*. USENIX Association, USA, 209–224.

[11] Sergiu Carpov, Paul Dubrulle, and Renaud Sirdey. 2015. Armadillo: A Compilation Chain for Privacy Preserving Applications. In *Proceedings of the 3rd International Workshop on Security in Cloud Computing* (Singapore, Republic of Singapore) *(SCC '15)*. Association for Computing Machinery, New York, NY, USA, 13–19. doi:10.1145/2732516.2732520

[12] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. August 2016. TFHE: Fast Fully Homomorphic Encryption Library. https://tfhe.github.io/tfhe/.

[13] Meghan Cowan, Deeksha Dangwal, Armin Alaghi, Caroline Trippel, Vincent T. Lee, and Brandon Reagen. 2021. Porcupine: A Synthesizing Compiler for Vectorized Homomorphic Encryption. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) *(PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 375–389. doi:10.1145/3453483.3454050

[14] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: An Encrypted Vector Arithmetic Language and Compiler for Efficient Homomorphic Computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) *(PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 546–561. doi:10.1145/3385412.3386023

[15] Sun Ding and Hee Beng Kuan Tan. 2013. Detection of Infeasible Paths: Approaches and Challenges. In *Evaluation of Novel Approaches to Software Engineering*, Leszek A. Maciaszek and Joaquim Filipe (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 64–78.

[16] Vivian Ding, Coşku Acay, and Andrew C. Myers. 2023. An Array Intermediate Language for Mixed Cryptography. In *FCS*.

[17] Jeanne Ferrante and Mary Mace. 1985. On Linearizing Parallel Code. In *Proceedings of the 12th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (New Orleans, Louisiana, USA) *(POPL '85)*. Association for Computing Machinery, New York, NY, USA, 179–190. doi:10.1145/318593.318636

[18] Shruthi Gorantala, Rob Springer, Sean Purser-Haskell, William Lam, Royce Wilson, Asra Ali, Eric P. Astor, Itai Zukerman, Sam Ruth, Christoph Dibak, Phillipp Schoppmann, Sasha Kulankhina, Alain Forget, David Marn, Cameron Tew, Rafael Misoczki, Bernat Guillen, Xinyu Ye, Dennis Kraft, Damien Desfontaines, Aishe Krishnamurthy, Miguel Guevara, Irippuge Milinda Perera, Yurii Sushko, and Bryant Gipson. 2021. A General Purpose Transpiler for Fully Homomorphic Encryption. Cryptology ePrint Archive, Paper 2021/811. https://eprint.iacr.org/2021/811. https://eprint.iacr.org/2021/811

[19] Charles Gouert, Vinu Joseph, Steven Dalton, Cedric Augonnet, Michael Garland, and Nektarios Georgios Tsoutsos. 2023. Accelerated Encrypted Execution of General-Purpose Applications. Cryptology ePrint Archive, Paper 2023/641. https://eprint.iacr.org/2023/641. https://eprint.iacr.org/2023/641

[20] Shai Halevi and Victor Shoup. 2012. Design and Implementation of a Homomorphic-Encryption Library.

[21] Christoph A. Herrmann and Tobias Langhammer. 2006. Combining partial evaluation and staged interpretation in the implementation of domain-specific languages. *Science of Computer Programming* 62, 1 (2006), 47–65. Special Issue on the First MetaOCaml Workshop 2004. doi:10.1016/j.scico.2006.02.002

[22] Seungwan Hong, Seunghong Kim, Jiheon Choi, Younho Lee, and Jung Hee Cheon. 2021. Efficient Sorting of Homomorphic Encrypted Data with k-way Sorting Network. Cryptology ePrint Archive, Paper 2021/551. https://eprint.iacr.org/2021/551. doi:10.1109/TIFS.2021.3106167

[23] Neil D. Jones. 1996. An introduction to partial evaluation. *ACM Comput. Surv.* 28, 3 (Sept. 1996), 480–503. doi:10.1145/243439.243447

[24] Yunsup Lee, Ronny Krashinsky, Vinod Grover, Stephen W. Keckler, and Krste Asanović. 2013. Convergence and scalarization for data-parallel architectures. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 1–11. doi:10.1109/CGO.2013.6494995

[25] Raghav Malik, Kabir Sheth, and Milind Kulkarni. 2023. Coyote: A Compiler for Vectorizing Encrypted Arithmetic Circuits. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 118–133. doi:10.1145/3582016.3582057

[26] Raghav Malik, Vidush Singhal, Benjamin Gottfried, and Milind Kulkarni. 2021. Vectorized Secure Evaluation of Decision Forests. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) *(PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 1049–1063. doi:10.1145/3453483.3454094

[27] Simon Moll and Sebastian Hack. 2018. Partial Control-Flow Linearization. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Philadelphia, PA, USA) *(PLDI 2018)*. Association for Computing Machinery, New York, NY, USA, 543–556. doi:10.1145/3192366.3192413

[28] Simon Moll and Sebastian Hack. 2018. Partial Control-Flow Linearization. *SIGPLAN Not.* 53, 4 (jun 2018), 543–556. doi:10.1145/3296979.3192413

[29] Umut Sahin. 2013. Encrypted Key-value Database Using Homomorphic Encryption. https://www.zama.org/post/encrypted-key-value-database-using-homomorphic-encryption

[30] Charitha Saumya, Kirshanthan Sundararajah, and Milind Kulkarni. 2022. DARM: Control-Flow Melding for SIMT Thread Divergence Reduction. In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization* (Virtual Event, Republic of Korea) *(CGO '22)*. IEEE Press, 28–40. doi:10.1109/CGO53902.2022.9741285

[31] Ryan Taylor and Xiaoming Li. 2011. Software-Based Branch Predication for AMD GPUs. *SIGARCH Comput. Archit. News* 38, 4 (jan 2011), 66–72. doi:10.1145/1926367.1926379

[32] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. 2023. HECO: Fully Homomorphic Encryption Compiler. arXiv:2202.01649 [cs.CR]

[33] Kirsten Winter, Chenyi Zhang, Ian J. Hayes, Nathan Keynes, Cristina Cifuentes, and Lian Li. 2013. Path-Sensitive Data Flow Analysis Simplified. In *Formal Methods and Software Engineering*, Lindsay Groves and Jing Sun (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 415–430.

[34] Qianchuan Ye and Benjamin Delaware. 2022. Oblivious algebraic data types. *Proc. ACM Program. Lang.* 6, POPL, Article 51 (Jan. 2022), 29 pages. doi:10.1145/3498713

[35] Qianchuan Ye and Benjamin Delaware. 2023. Taype: A Policy-Agnostic Language for Oblivious Computation. *Proc. ACM Program. Lang.* 7, PLDI, Article 147 (June 2023), 25 pages. doi:10.1145/3591261

A Evaluation rules for PF

$$\frac{\overline{x}\colon \overline{\tau} \vdash e : \texttt{int}\ \ e \longrightarrow^{*} n}{\lambda(\overline{x}\colon \overline{\tau}).\ (p,\ e) \rightsquigarrow \lambda(\overline{x}\colon \overline{\tau}).\ (p,\ n)}\ \textsc{Pf-Eval} \qquad \frac{p \Longrightarrow (e \equiv n)}{(p,\ e) \rightsquigarrow (p,\ n)}\ \textsc{Pf-Spec} \qquad \frac{r_i \rightsquigarrow r_i';\ \ldots}{r_i;\ldots \rightsquigarrow r_i';\ldots}\ \textsc{Pf-Concat}$$

$$\frac{p \Longrightarrow b}{(p,\ \texttt{if}\ b\ \texttt{then}\ e_1\ \texttt{else}\ e_2) \rightsquigarrow (p,\ e_1)}\ \textsc{Pf-IfT} \qquad \frac{p \Longrightarrow \neg b}{(p,\ \texttt{if}\ b\ \texttt{then}\ e_1\ \texttt{else}\ e_2) \rightsquigarrow (p,\ e_2)}\ \textsc{Pf-IfF}$$

$$\frac{}{(p,\ \texttt{if}\ b\ \texttt{then}\ e_1\ \texttt{else}\ e_2) \rightsquigarrow (p \wedge b,\ e_1);\ (p \wedge \neg b,\ e_2)}\ \textsc{Pf-IfB}$$

$$\frac{(p,\ e_1) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ \texttt{let}\ x = e_1\ \texttt{in}\ e_2) \rightsquigarrow (p_i',\ \texttt{let}\ x = e_i'\ \texttt{in}\ e_2)}\ \textsc{Pf-Let1} \qquad \frac{}{(p,\ \texttt{let}\ x = ne\ \texttt{in}\ e) \rightsquigarrow (p,\ e[x \mapsto ne])}\ \textsc{Pf-Let2}$$

$$\frac{(p,\ e_1) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ e_1 \oplus e_2) \rightsquigarrow (p_i',\ e_i' \oplus e_2);\ldots}\ \textsc{Pf-ArithL} \qquad \frac{(p,\ e_2) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ ne \oplus e_2) \rightsquigarrow (p_i',\ ne \oplus e_i');\ldots}\ \textsc{Pf-ArithR}$$

$$\frac{(p,\ e_1) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3) \rightsquigarrow (p_i',\ \texttt{if}\ e_i'\ \texttt{then}\ e_2\ \texttt{else}\ e_3);\ldots}\ \textsc{Pf-IfCond}$$

$$\frac{(p,\ e_2) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ e_1[e_2]) \rightsquigarrow (p_i',\ e[e_i']);\ldots}\ \textsc{Pf-ArrLookup1} \qquad \frac{n \le k}{(p,\ [e_1,\ldots,e_k][n]) \rightsquigarrow (p,\ e_n)}\ \textsc{Pf-ArrLookup2}$$

$$\frac{n > k}{(p,\ [e_1,\ldots,e_k][n]) \rightsquigarrow (p,\ 0)}\ \textsc{Pf-ArrLookup3} \qquad \frac{(p,\ e_1) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ e_1[ee_2 \rightarrow ee_3]) \rightsquigarrow (p_i',\ e_i'[ee_2 \rightarrow ee_3])}\ \textsc{Pf-ArrUpdate1}$$

$$\frac{(p,\ ee_2) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ [e_1,\ldots,e_k][ee_2 \rightarrow ee_3]) \rightsquigarrow (p_i',\ [e_1,\ldots,e_k][e_i' \rightarrow ee_3])}\ \textsc{Pf-ArrUpdate2}$$

$$\frac{n < k}{(p,\ [e_1,\ldots,e_k][n \rightarrow ee_3]) \rightsquigarrow (p,\ [e_1,\ldots\widehat{e_n},ee_3,\ldots,e_k])}\ \textsc{Pf-ArrUpdate3}$$

$$\frac{(p,\ e_i) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ [e_1,\ldots,e_k]) \rightsquigarrow (p_i',\ [e_1,\ldots,e_i',\ldots,e_k]);\ldots}\ \textsc{Pf-Arr} \qquad \frac{\texttt{fn}\ f(x\colon \tau) = e_2}{(p,\ f\ e_1) \rightsquigarrow (p,\ e_2[x \mapsto e_1])}\ \textsc{Pf-App}$$

$$\frac{(p,\ e_1) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ \texttt{mux}\ e_1\ e_2\ e_3) \rightsquigarrow (p_i',\ \texttt{mux}\ e_i'\ e_2\ e_3);\ldots}\ \textsc{Pf-Mux1} \qquad \frac{(p,\ e_2) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ \texttt{mux}\ b\ e_2\ e_3) \rightsquigarrow (p_i',\ \texttt{mux}\ b\ e_i'\ e_3);\ldots}\ \textsc{Pf-Mux2}$$

$$\frac{(p,\ e_3) \rightsquigarrow (p_i',\ e_i');\ldots}{(p,\ \texttt{mux}\ b\ ne\ e_3) \rightsquigarrow (p_i',\ \texttt{mux}\ b\ ne\ e_i');\ldots}\ \textsc{Pf-Mux3}$$

$$\frac{\Gamma \vdash e_1 : \underline{\texttt{int}}\ \ \Gamma \vdash e_2 : \underline{\texttt{int}}}{(p_1,\ e_1);\ (p_2,\ e_2) \rightsquigarrow (p_1 \vee p_2,\ [\![p_1\ ?\ e_1 : e_2]\!])}\ \textsc{Pf-MergeInt} \qquad \frac{\Gamma \vdash e_1 : \overline{\mathcal{A}^{\top}}\ \ \Gamma \vdash e_2 : \overline{\mathcal{A}^{\top}}}{(p_1,\ e_1);\ (p_2,\ e_2) \rightsquigarrow (p_1 \vee p_2,\ [\![p_1\ ?\ e_1 : e_2]\!])}\ \textsc{Pf-MergeArr}$$

$$[\![b\ ?\ e : e]\!] \rightsquigarrow e$$

$$[\![b\ ?\ e_1 \oplus e_2 : e_1 \oplus' e_3]\!] \rightsquigarrow e_1 \oplus [\![b\ ?\ e_2 : e_3]\!] \qquad \text{when } \oplus \equiv \oplus'$$

$$[\![b\ ?\ e_1 \oplus e_2 : e_3 \oplus' e_2]\!] \rightsquigarrow [\![b\ ?\ e_1 : e_3]\!] \oplus e_2 \qquad \text{when } \oplus \equiv \oplus'$$

$$[\![b\ ?\ [e_i,\ldots] : [e_i',\ldots]]\!] \rightsquigarrow [\ [\![b\ ?\ e_i : e_i']\!],\ldots\ ] \qquad \text{when only one of } e_i \neq e_i'$$

$$[\![b\ ?\ e_1 : e_2]\!] \rightsquigarrow \texttt{mux}(b,\ e_1,\ e_2) \qquad \text{when no other rule applies}$$

B Proofs and additional lemmas

Proof of Theorem 4.3. Let p be a well-typed path forest not in normal form. By definition it must contain some well-typed Coil expression e also not in normal form. We proceed by induction over e: If e fails to be in normal form because of an unreduced subexpression, then by the inductive hypothesis we can step the subexpression. In the cases where all subexpressions of e are in normal form:

1. For a `let` expression we can apply the Pf-Let2 rule
2. For an `if` statement, we can apply either Pf-IfT, Pf-IfF, or Pf-IfB
3. For an array update, we can apply Pf-ArrUpdate3

4. For an array index, we can apply Pf-ArrLookup2 or Pf-ArrLookup3, depending on whether the index is in bounds or not

Thus, either r is in normal form, or $r \rightsquigarrow r'$. □

Lemma B.1. *Let* $\lambda(\overline{x}: \overline{\tau}).\ (p,\ e)$ *be a well-typed path forest. Then for every set of ciphertext inputs* $v_1, \ldots, v_n$*, if*

$$p(v_1, \ldots, v_n) \longrightarrow^* \texttt{false}$$

and $\lambda(\overline{x}: \overline{\tau}).\ (p,\ e) \rightsquigarrow \lambda(\overline{x}: \overline{\tau}).\ (p_i',\ e_i');\ldots$*, then* $\forall\ i.\ p_i'(v_1, \ldots, v_n) \longrightarrow^* \texttt{false}$.

Proof. This follows immediately by induction on $\cdot \rightsquigarrow \cdot$. □

Lemma B.2 (Well-formed)**.** *If* r *is a path forest such that* $\Gamma \vdash \texttt{WF}(r)$ *and* $r \rightsquigarrow r'$*, then* $\Gamma \vdash \texttt{WF}(r')$.

Proof. It is clear that r' is well-typed from Theorem 4.3. To show that it is deterministic, fix some ciphertext inputs $\overline{v}$, and proceed by induction on $\rightsquigarrow$.

Most of the cases are relatively uninteresting: either the set of paths does not change after stepping, or the new set of paths is well-formed by the inductive hypothesis. The remaining cases are PF-Concat and PF-IfB:

1. PF-Concat: By inversion, $r \rightsquigarrow r'$ has the form

   $$(p_i,\ e_i);\ldots \rightsquigarrow (p_i',\ e_i');\ldots$$

   Then by the inductive hypothesis, for each

   $$p_i(\overline{v}) \longrightarrow^* \texttt{true}$$

   the corresponding

   $$p_i'(\overline{v}) \longrightarrow^* \texttt{true}$$

   Lemma B.1 tells us the inverse. That r' is well-formed follows immediately.
2. Pf-IfB: By inversion, $r \rightsquigarrow r'$ has the form

   $$(p,\ \texttt{if}\ b\ \texttt{then}\ e_1\ \texttt{else}\ e_2) \rightsquigarrow (p \wedge b,\ e_1);\ (p \wedge \neg b,\ e_2)$$

   By well-formedness of r we may further assume

   $$p(\overline{v}) \longrightarrow^* \texttt{true}$$

   Now, note that exactly one of $b(\overline{v})$ and $\neg b(\overline{v})$ evaluates to $\texttt{false}$; thus

   $$(p \wedge b,\ e_1);\ (p \wedge \neg b,\ e_2)$$

   must also be well-formed.

Thus, $\Gamma \vdash \texttt{WF}(r')$ □

Proof of Theorem 4.4. By Lemma B.2, $r'(\overline{v}) \longrightarrow^* v$ is a judgement that makes sense. We now proceed by induction on $\rightsquigarrow$. Most of the cases are, again, uninteresting as they do not change the set of paths, correspond directly to an evaluation rule from Taype. We again show the cases PF-Concat and PF-IfB:

- Pf-Concat: By assumption, $r = (p_i,\ e_i);\ldots$ is well-formed so we can take $p_i(\overline{v}) \longrightarrow^* \texttt{true}$ and $r(\overline{v}) \longrightarrow e_i(\overline{v})$ By Lemmas B.1 and B.2, $r' \longrightarrow e_i'(\overline{v})$, and by the inductive hypothesis $e_i'(\overline{v}) \longrightarrow^* v$ so we are done.
- Pf-IfB: Let $r = (p,\ \texttt{if}\ b\ \texttt{then}\ e_1\ \texttt{else}\ e_2)$.
  By well-formedness of r we once again have that $p(\overline{v}) \longrightarrow^* \texttt{true}$ so $r' = (b,\ e_1);\ (\neg b,\ e_2)$.
  Proceed by case-analysis on b:
  - When $b(\overline{v}) \longrightarrow^* \texttt{true}$, $r'(\overline{v}) \longrightarrow e_1(\overline{v})$ and

    $$\texttt{if}\ b\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \longrightarrow^* \texttt{mux}\ \texttt{true}\ e_1\ e_2 \longrightarrow e_1$$

  - When $b(\overline{v}) \longrightarrow^* \texttt{false}$, $r'(\overline{v}) \rightarrow e_2(\overline{v})$ and

    $$\texttt{if}\ b\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \longrightarrow^* \texttt{mux}\ \texttt{false}\ e_1\ e_2 \longrightarrow e_2$$

□

C Full Implementations of Benchmarks

Linear index:

```
fn lookup(arr: 𝒜^ℓ, start: int, end: int, index: int): int =
  if start ≡ end then 0
  else if start ≡ index then arr[start]
  else lookup(arr, start + 1, end, index)
```

Logarithmic index:

```
fn halfhelper(x: int, i: int): int =
  if x < i * 2 then i - 1 else halfhelper(x, i + 1)
fn half(x: int): int = halfhelper(x, 0)
fn lookup(array: 𝒜^ℓ,
          start: int, end: int, index: int): int =
  if start ≡ end - 1 then array[start]
  else
    let mid = start + half(end - start) in
    if index < mid then lookup(array, start, mid, index)
    else lookup(array, mid, end, index)
```

Associative array:

```
fn lookup(keys: 𝒜^ℓ, values: 𝒜^ℓ,
          i: int, size: int, key: int): int =
  if i ≡ size then 0
  else if keys[i] ≡ key then values[i]
  else lookup(keys, values, i + 1, size, key)
```

Filter:

```
fn predicate(val: int, threshold: int): int =
  if val < threshold then 0 else val
fn filter(arr: 𝒜^ℓ,
          cur: int, len: int, threshold: int): 𝒜^ℓ =
  if cur ≡ len then arr
  else
    filter(arr[cur → predicate(arr[cur], threshold)],
           cur + 1, len, threshold)
```

Second-price auction:

```
fn auction(bids: 𝒜^ℓ̄, cur: int, len: int,
           id1: int, id2: int): 𝒜^⊤,⊤ =
  if cur ≡ len then [ ↑id1, bids[id2] ]
  else if bids[id1] < bids[cur] then
    auction(bids, cur + 1, len, cur, id1)
  else
    auction(bids, cur + 1, id1, id2)
```

Merge:

```
fn merge(arr1: 𝒜^ℓ̄, i1: int, len1: int,
         arr2: 𝒜^ℓ̄, i2: int, len2: int,
         out: 𝒜^ℓ̄, j: int): 𝒜^ℓ̄ =
  if i1 ≡ len1 then
    if i2 ≡ len2 then out
    else merge(arr1, i1, len1,
               arr2, i2 + 1, len2,
               out[ j ⟶ arr2[i2] ], j + 1)
  else
    if i2 ≡ len2 then merge(arr1, i1 + 1, len1,
                            arr2, i2, len2,
                            out[ j ⟶ arr1[i1] ])
    else if arr1[i1] < arr2[i2] then
      merge(arr1, i1 + 1, len1,
            arr2, i2, len2,
            out[ j ⟶ arr1[i1] ], j + 1)
    else
      merge(arr1, i1, len1,
            arr2, i2 + 1, len2,
            out[ j ⟶ arr2[i2] ], j + 1)
```